MAS: A durable communication protocol for auditable multi-agent workflows

Emmett Miller

Abstract

MAS is an open-source Rust implementation of a bounded communication protocol for multi-agent workflows. A versioned text envelope, durable mailbox and optional task profile separate message receipt from task completion. An assessment profile binds findings to exact candidates, evaluators, rubrics and evidence. The implementation provides identity-preserving retries, explicit routes, resource limits and runtime-observed expiry. We describe its representation, admission rules and integration boundary, and evaluate executable tests and finite Promela models with SPIN. The evidence supports these communication and assessment controls under the stated storage, host and model assumptions. Source, models and reproduction scripts are available.

Keywords: multi-agent systems, communication protocol, durable messaging, model checking, auditable assessment

Code metadata

	Description	Value
C1	Current code version	Package 0.1.0. Examined revision `a61b3da98ef3ef11996dd79b4a3a048ed6c98ce1`
C2	Permanent link to code	https://github.com/emmett08/mas/tree/a61b3da98ef3ef11996dd79b4a3a048ed6c98ce1
C3	Legal code licence	GPL-3.0-only with third-party notices
C4	Versioning system	Git
C5	Languages, tools and services	Rust, nom and SQLite. Promela/SPIN and a C compiler support verification. Python supports optional host integration.
C6	Compilation and environment	Pinned Rust 1.99.0, Cargo.lock and bundled SQLite. The verification harness runs on Linux. Optional PostgreSQL support is feature-gated.
C7	Developer documentation	https://github.com/emmett08/mas/blob/a61b3da98ef3ef11996dd79b4a3a048ed6c98ce1/README.md
C8	Support channel	https://github.com/emmett08/mas/issues

1. Motivation and significance

An agent workflow needs to distinguish an accepted message, a completed task and a correct result. After losing a response, the caller must distinguish delivery, execution and reassessment before choosing a retry. MAS provides explicit records for these different questions. Its contribution is a reusable implementation combining bounded admission, durable identity retention, task correlation and content-bound assessments, with executable tests and finite-state models.

Agent2Agent (A2A) defines interoperable agent tasks, messages, artefacts and discovery. The Model Context Protocol (MCP) connects hosts to tools, resources and prompts [1, 2]. MAS addresses the local admission and retention of workflow exchanges. A2A extensions and application-specific MCP services can carry these workflow contracts. Their deployment must supply the corresponding persistence and recovery behaviour. MAS complements those interoperability interfaces with explicit local state transitions.

2. Software description

2.1. Protocol design

Holzmann specifies a protocol through five elements: its service, execution environment, message vocabulary, encoding and exchange rules [3, Section 2.2, p. 21]. MAS gives each element a distinct role in its communication contract.

The service defines the observable mailbox behaviour. A send atomically inserts an accepted message identity and its content. An identical retry recovers the original acceptance. Receive returns addressed, unacknowledged messages in insertion order and records receipt. Acknowledgement removes a received message from the pending set while preserving its identity and history. These operations give callers a stable recovery contract across retries.

The environment specifies the premises supporting that contract. SQLite durability depends on functioning filesystem locks and flushes. Protected credentials and host isolation support actor binding. Deadline observation requires a host clock and continued execution. Within those premises, the gateway derives the actor from a capability, while the store enforces membership, routes, budgets and mailbox capacity. Separating environmental premises from enforced checks identifies which component must sustain each property.

The vocabulary identifies admissible message meanings. The envelope's `kind` field permits application-defined names. The `agent-task/1` profile selects `Request`, `Result`, `Failure` and `Cancel`, with specified payloads. The operations `send`, `receive` and `ack` belong to the mailbox service and the gateway's command vocabulary. Their distinct representations preserve the difference between delivering a message and reporting a task outcome.

The encoding represents messages as bounded UTF-8 text with seven ordered fields and a JSON object payload whose numeric values are integers. Complete-input parsing checks syntax, identifiers, unique object keys and size limits. Canonical content provides a stable comparison for retries across equivalent accepted spellings. The application profile then assigns task meaning to the decoded values.

The exchange rules relate each operation to retained history. Replies identify an exact request and reverse its participants. Result and failure require recorded receipt, and a terminal transition requires an open task. Validation, insertion and the task update share a write transaction, which orders concurrent terminal proposals by committed state. Parser tests therefore examine representation, repository tests exercise admission, and finite models explore exchanges under explicit fault assumptions. This division makes dependencies separately inspectable, following Holzmann's modular design argument [3, Section 2.7, pp. 36–37].

2.2. Software architecture

The examined revision [4] implements the codec, SQLite repository, task transitions and capability gateway in Rust. The `nom` parser recognises the complete envelope before values enter the repository. The optional Python host delegates mailbox operations to the Rust boundary. The evaluation concerns this native implementation with its default SQLite backend. Optional PostgreSQL support and other host integrations provide further deployment choices.

Figure 1 separates logical communication permission from physical transport. A directed graph determines which registered participants may exchange messages. Star, mesh and tree configurations use the same central repository. A mesh permits peer-addressed messages through that repository. The gateway derives the run and actor from host-provisioned capabilities. Direct repository and administrative CLI calls trust the supplied actor, so the host must restrict database and administrative access to trusted components.

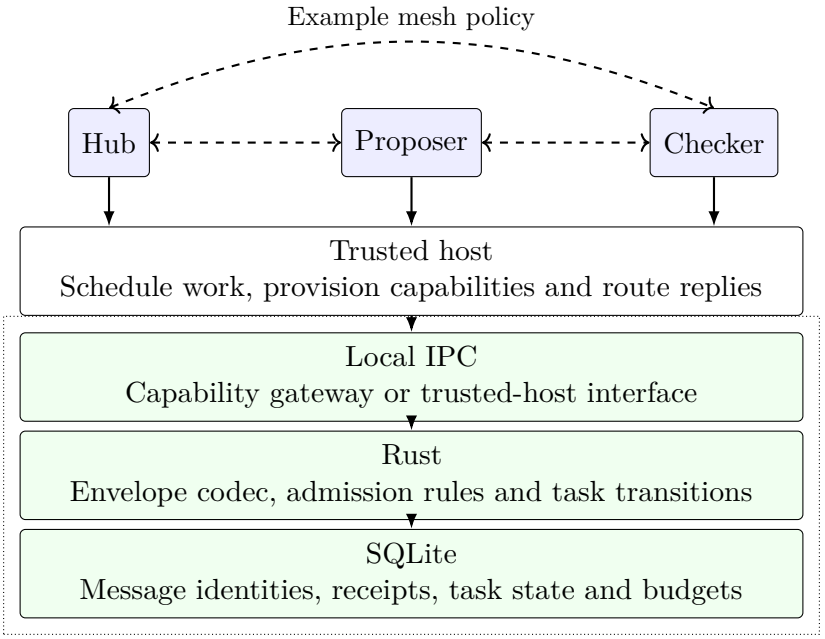


Figure 1: Logical routes and the local implementation boundary. Dashed links indicate permitted exchanges. Solid arrows indicate requests to the central implementation. The host schedules work and controls external operations. The repository transaction governs mailbox and task state. Diagram source prepared with ChatGPT (OpenAI, model version unavailable).

Local interprocess communication (IPC) carries repository operations. The capability gateway uses JSON-line requests over standard input and output.

The persistent trusted-host interface has a JSON header and length-delimited body. The MAS envelope supplies the message representation carried by these operation interfaces. Host-provisioned capabilities bind gateway operations to an actor and run.

2.3. Software functionalities

2.3.1. Representation and identity

Version 1 uses seven ordered fields: message identifier, run identifier, sender, recipient, version, kind and payload. The payload is a JSON object with an exact-integer subset of JSON's number syntax [5]. The parser rejects trailing input, duplicate decoded keys, malformed Unicode, fractions and exponent notation. Limits are 65,536 UTF-8 bytes per envelope, 32 container levels and 1,024 decimal digits per integer. Applications can represent fractions as declared rational records or decimal strings.

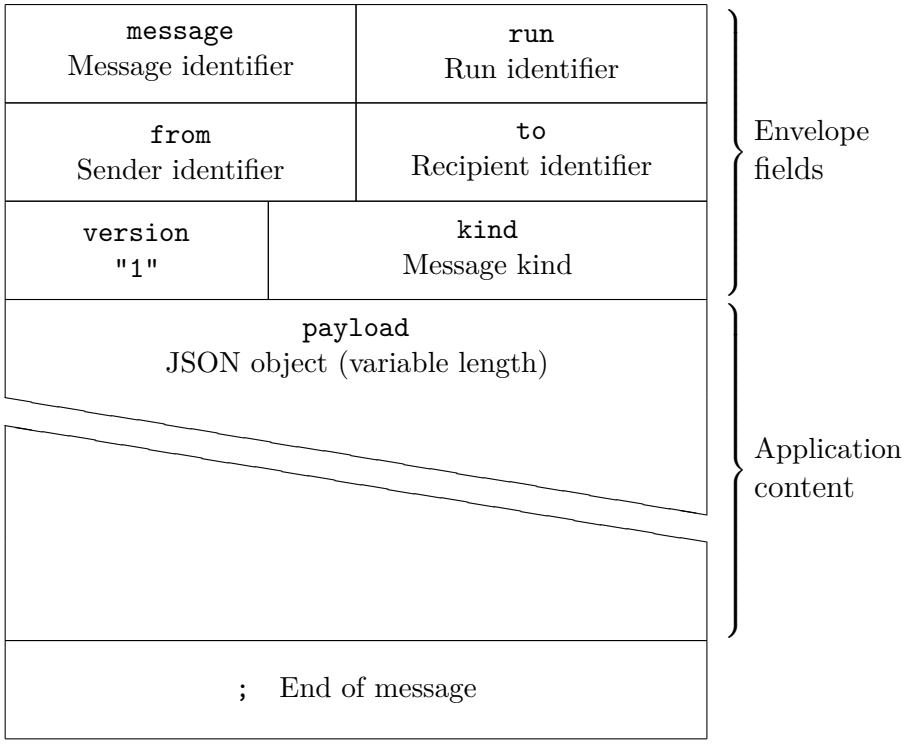
Figure 2 groups the envelope fields in their required order. Their encoded lengths vary with their values. Keywords, spaces, quotation marks and the final semicolon delimit the canonical text. The canonical encoder normalises whitespace and object-key order. Message identity is the pair (r, i) of run and message identifiers. The repository retains the complete canonical envelope representation. Equal content under an existing identity is an identical retry, whereas changed content is a conflict. The identity remains retained after acknowledgement.

2.3.2. Admission, delivery and task state

A run freezes membership, routes, application profile, mailbox capacity, communication and call budgets, and a deadline. Algorithm 1 summarises SQLite admission. Identical retries retain their original acceptance even after the deadline or when the mailbox is full. New sends charge the byte budget using the stored JSON envelope representation. The text-wire length limit applies separately. A full mailbox retains accepted messages and rejects new admissions for later retry.

The pseudocode's requirements return errors on failure. An uncommitted write transaction rolls back. Independent expiry reconciliation and gateway attempt accounting can already have committed. Database failures yield an unknown admission outcome in the typed Rust API. The caller must reconcile the original identity because an outstanding operation may commit after an observation. These semantics rely on SQLite's transaction and storage assumptions [6, 7].

Receive returns addressed, unacknowledged messages in insertion order and records receipt. Repeated receive can return the same message until acknowledgement. The optional `agent-task/1` profile additionally correlates



Field order: left to right, then top to bottom. Widths are schematic.

Canonical example (76 octets):

```
message "q" run "r" from "a" to "b"
version "1" kind "Info" payload {"n":3};
```

A single space joins the displayed lines. The semicolon ends the message.

Figure 2: MAS envelope structure and canonical example. The six leading values are quoted strings, followed by a JSON object payload. Fields appear in the displayed order. Box widths indicate logical grouping. Encoded field lengths depend on their values, up to 65,536 UTF-8 bytes for the entire envelope. Diagram source prepared with ChatGPT (OpenAI, model version unavailable).

each result or failure with one received request, reverses its participants and requires an open task. A Result reports either `completed` or `unresolved`. Both close the task and record a worker-reported outcome. Application checks assess output correctness. Cancellation comes from the requester. Result, failure, cancellation and observed expiry compete for the first committed terminal transition (Figure 3). Receipt permits a reply before acknowledgement.

Expiry reconciliation closes open tasks as `expired_outcome_unknown`, even when message capacity is exhausted. Its observation time depends on host activity and clock and database availability. A closed request can remain

Algorithm 1 Admit a message into the SQLite repository

Input: Typed message m, host-bound actor a, repository D
Output: Original receipt, new receipt, or typed admission error

Require an existing run and reconcile observed expiry
Encode and reparse m to derive canonical stored content c
Begin an immediate write transaction and load frozen run r
Require registered $a = m$.sender and recipient, with an allowed route
if D contains $(m.\text{run}, m.\text{id})$ **then**
    Require retained content $= c$
    Commit and **return** original sequence with duplicate flag
end if
Reconcile the deadline again under the write transaction
if the run has expired **then**
    Commit expiry and **return** deadline rejection
end if
Require active version, remaining message/byte budgets and mailbox space
if the run uses the task profile **then**
    Require a valid transition from the retained task state
end if
Insert c, record any task transition and charge budgets
Commit and **return** new sequence with duplicate flag cleared

in the mailbox as history with `actionable:false`, a snapshot of task state at receipt. Hosts must arbitrate concurrent execution and derive permissions from trusted configuration. Applications establish external-effect outcomes through their own execution and observation controls. This division follows the end-to-end distinction between lower-level delivery and application-level completion [8].

2.3.3. Auditable assessments

The `assessment.v1` profile adds typed payload records within the version-1 grammar. Candidate, evidence, rubric and assessment records have explicit identities and schema versions. References bind both an identifier and a SHA-256 digest of the complete referenced record. Separate output and evidence-content digests bind the contained values. Canonicalisation preserves exact integers, array order and Unicode sequences, and sorts object keys by Unicode scalar value. This profile-specific form differs from RFC 8785, which uses ECMAScript number serialisation and UTF-16 code-unit key order [9].

The contextual validator in Algorithm 2 checks authority and reference

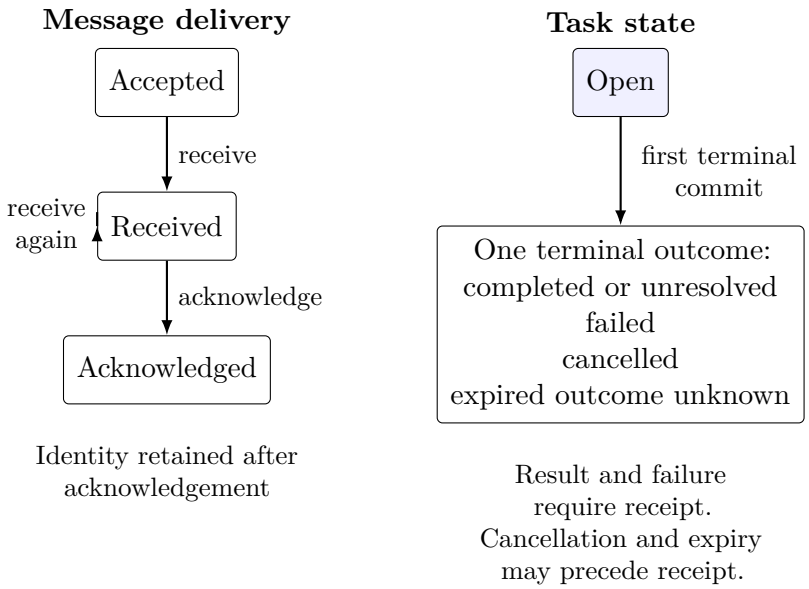


Figure 3: Two independent state records for a request. Receive enables a worker reply. Acknowledgement releases mailbox capacity. Expiry closes an open task through a retained state transition. External-effect outcomes require application-level observation. Diagram source prepared with ChatGPT (OpenAI, model version unavailable).

resolution after structural decoding. A completed assessment can contain failed criteria. An inconclusive finding carries a null score. An evaluator error carries an empty findings list. Scores are signed 64-bit integers within rubric bounds. The rubric and evaluator define their interpretation, including any acceptance threshold. The host must supply evaluator authority from its frozen invocation configuration, independently of the assessment payload.

An assessment travels inside the existing task Result's `output`. Its identity and evaluator must match the envelope's message identifier and sender. Revision validation requires a distinct candidate identity, an exact parent reference and precisely the supplied feedback references. An empty feedback set permits a resampling control. The in-memory `AssessmentHistory` deduplicates equal records and rejects changed content under the same identity. Durable revision reservations, disclosure policy and execution belong to a host integration. The protocol retains the supplied assessment and its references. Host disclosure controls govern evaluator access, while independent outcome checks assess judgement quality.

Algorithm 2 Validate an assessment against retained context

Input: Assessment A, candidate C, rubric R, evidence set E, host-authorised evaluator V

Output: Contextual validity or a specific validation error

Revalidate the structures of A, C, R, V
Require A.evaluator $= V$
Resolve A's candidate and rubric references against C, R
Require unique evidence identifiers and revalidate each record in E
if A.status $\neq$ error **then**
    Require one finding per rubric criterion
end if
for all findings f in A **do**
    Resolve criterion k in R and require k.measurement $= V$.kind
    Require each present score to lie within k's bounds
    for all evidence references e in f **do**
        Resolve e against E and compare its complete-record digest
    end for
end for
return valid

3. Illustrative examples

3.1. Retries and task closure

Figure 4 shows how a sender resolves uncertainty after losing an acceptance response. The insertion has committed, so an identical retry returns its existing sequence. The recipient then receives and acknowledges the retained message. These are host-level API interactions. An integration may forward them through the capability gateway, whose attempt budget also counts retries.

Task closure depends on the first committed terminal transition. In Figure 5, the worker receives a request and sends a completed Result before acknowledging that request. The Result closes the task. The later acknowledgement releases mailbox capacity. A subsequent Cancel is rejected. If cancellation commits first, a later Result is rejected instead. Application observations determine whether external work occurred.

3.2. Executed examples

The supplied star fixture registers a hub, proposer and checker. In a minimal sum exchange, the hub sends a request containing an operation and integer input. The worker receives it and returns a result correlated

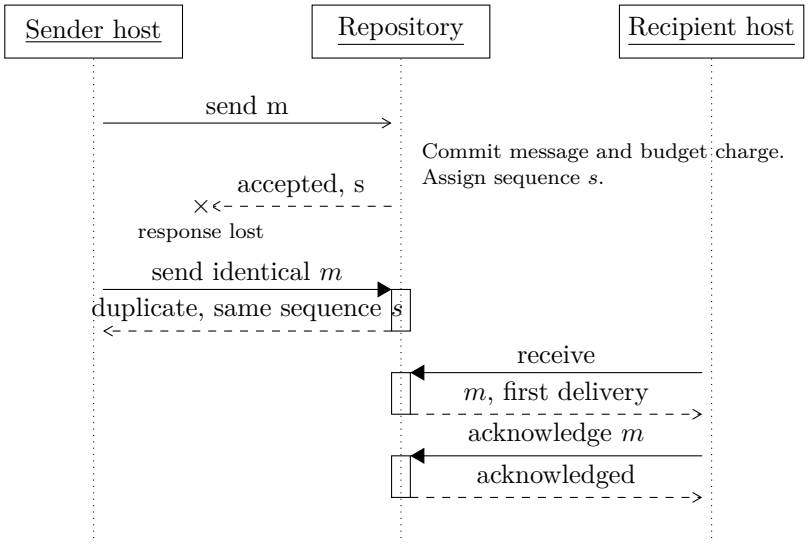


Figure 4: Recovery from a lost send response. The cross marks a response lost after commit. An identical retry returns the original sequence and preserves the original message and byte totals. The recipient's first receive remains a first delivery. Solid arrows show requests. Dashed arrows show responses. Diagram source prepared with ChatGPT (OpenAI, model version unavailable).

by `reply_to`. Restarting after receipt but before acknowledgement permits redelivery of the original request. An identical retry retains the original message record. Separate transition tests reject a different result attempting to close the same terminal task. The application checker compares the proposed sum with independently calculated arithmetic.

The repository's fixtures execute Rust parsing and SQLite exchanges locally. Their synthetic inputs and deterministic participants make the examples reproducible. The recovery example reopens storage and observes deadline expiry. The capability-gateway example challenges sender spoofing and retains attempt limits across restart. Table 2 reports the observed outcomes. Commands and exact output records accompany the paper.

3.3. Verification and reproducibility

The evaluation freezes revision `a61b3da` and combines implementation tests with selected executable abstractions. The reproduction script records source hashes, tool versions, commands and raw logs. The selected default-feature Rust tests exercise codecs, repository operations, gateway controls, observation and assessments against the local SQLite implementation.

SPIN checks assertions and temporal properties on finite Promela models [10]. Transport checks distinguish finite fault budgets from indefinitely

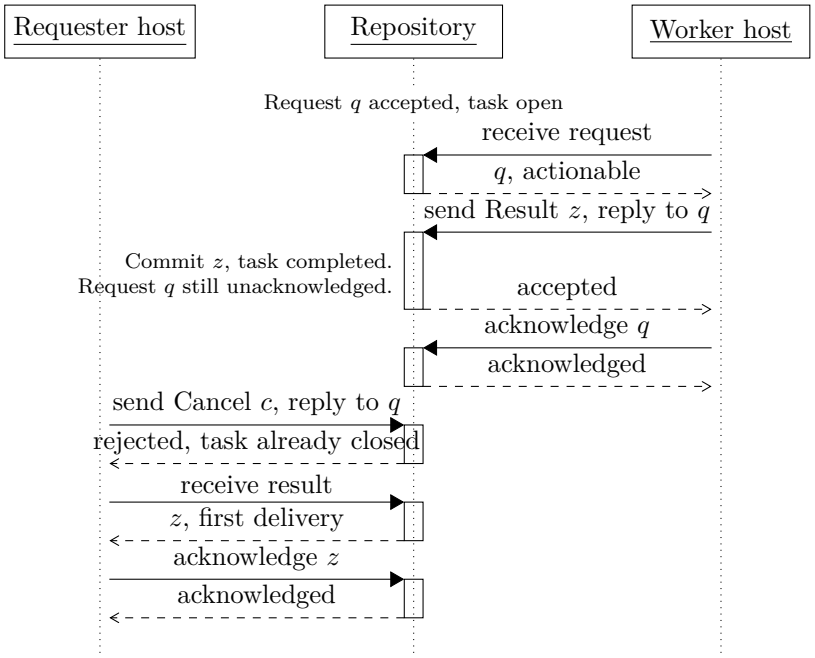


Figure 5: Task closure and delivery acknowledgement. The accepted request is q, the worker's completed Result is z, and c is a subsequent cancellation attempt. The exchange assumes authorised routes, available budgets and an unexpired run. Receipt permits the reply. Acknowledgement releases mailbox capacity. Diagram source prepared with ChatGPT (OpenAI, model version unavailable).

recurring faults. Safety concerns routing, identity, capacity and receipt order. Progress means eventual acknowledgement of each accepted message under a finite fault budget. Task, transaction and expiry models cover conflicting terminal outcomes and recovery boundaries. Assessment checks use two candidate identities, two assessment identities, two content values, two evaluators, three delivery attempts and two revision reservations. The model abstracts committed host transitions and digest values. Its disclosure and revision properties specify obligations for host integrations.

For each message identity i in the transport model, the progress requirement is

$$\mathsf{G}\Big(\text{accepted}_i \Rightarrow \mathsf{F}\,\text{acknowledged}_i\Big), \tag{1}$$

where G means always and F means eventually. Equation 1 states eventual acknowledgement of accepted messages under the model's assumptions. The checked star has three participants, two message identities and two content values, with capacities one and two. The bounded model permits each message one lost acknowledgement and one restart. The unlimited-fault variant permits recurring faults. Table 1 separates successful searches from

expected counterexamples.

Table 1: Recorded verification outcomes. Counterexamples and witnesses are distinct from positive searches.

Evidence category	Result
Executed Rust tests	79 passed, 0 failed
Ignored Rust model-suite entrypoints	3
Offline executable fixtures	3 completed
Complete model searches with zero errors	17
Replayed assertion-defect counterexamples	19
Replayed liveness-defect counterexamples	2
Unlimited-fault progress counterexample	1
Replayed reachability witnesses	2

Table 2: Observed offline fixture outcomes with deterministic local participants.

Fixture	Recorded observation
Star	4 accepted messages, 748 charged bytes and 0 call reservations. All messages acknowledged. Checker accepted the proposed sum.
Recovery	1 charged message and 1 task expired with unknown outcome. Historical delivery marked non-actionable. Identical retry retained sequence 1.
Gateway	4 charged attempts, including a rejected sender forgery. Message received and acknowledged after restart. Restart preserved the exhausted attempt budget.

The 17 October 2026 run used Rust/Cargo 1.99.0, SPIN 6.5.2 and GCC 13.3.0. Each model command had a 30-second, 512-MiB limit and depth 100,000. Successful verification searches exhaustively explored the specified finite abstractions. Defect counterexamples demonstrate detection of the introduced faults. Under indefinitely recurring faults, the transport model contains an execution in which an accepted message remains pending indefinitely. Witnesses establish reachability. These results support the declared communication and assessment controls, with selected implementation tests supplying correspondence evidence.

Negative controls introduce isolated defects into model copies. Their intended assertions or acceptance cycles must be detected and replayed. Reachability witnesses establish that accepted assessments and successful revision paths are possible. Witness failures are deliberately constructed observations, distinct from successful safety searches. Searches that exceed a resource bound are inconclusive. Selected implementation tests supply

correspondence evidence between Rust behaviour and the models. Model state counts quantify the explored abstract state spaces.

4. Impact

MAS provides an experimental substrate for comparing agent workflows while retaining who sent each result, its task correlation and the assessment context. Researchers can vary logical routes while preserving envelope semantics and can distinguish a delivery failure from a rejected substantive answer. Exact candidate and feedback references support comparisons between revision and budget-matched resampling. Such comparisons require independent outcome measures, frozen evaluators and protected held-out cases. The present evaluation measures communication and assessment controls.

The implementation is appropriate when bounded work, durable retries and inspectable task histories justify a central authority. A direct function call is simpler for one synchronous trusted component. A conventional queue may suffice when task correlation and assessment records already exist elsewhere. A2A is relevant when independently hosted agents need a standard external interface. MCP is relevant when clients need standard tool access. The required communication and authority boundaries determine the integration choice.

Storage durability, trusted-host isolation and correct adapter behaviour remain premises. Cancellation closes the protocol task. External effects require application-specific observation and, where applicable, compensation. The present evidence covers the selected local implementation and finite models. Deployment qualification addresses scale, remote security, long-duration availability and comparative throughput.

5. Conclusions

MAS separates communication permission, durable delivery, task closure and assessment validity. Its Rust implementation makes retries and terminal transitions explicit, while its assessment profile binds findings to exact retained context. Executable tests and model checking of finite abstractions support these controls within stated assumptions. The retained records provide inputs for reproducible experiments measuring the effects of feedback.

Data and software availability

The source revision in the metadata table contains the implementation, tests and Promela models. The accompanying `paper/` directory contains

the manuscript, vector figure sources, reproduction script and retained verification evidence. The fixtures use synthetic inputs and deterministic local participants.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

ChatGPT (OpenAI, model version unavailable) assisted with source inspection, literature searches, manuscript drafting and editing, diagram preparation and reproduction-script development.

References

[1] A2A Project, Agent2Agent (A2A) protocol specification, release v1.0.1, 28 May 2026, accessed 17 October 2026 (2026).
URL https://github.com/a2aproject/A2A/blob/v1.0.1/docs/specification.md

[2] Model Context Protocol contributors, Model Context Protocol: Specification, protocol revision 2025-11-25, accessed 17 October 2026 (2025).
URL https://modelcontextprotocol.io/specification/2025-11-25

[3] G. J. Holzmann, Design and Validation of Computer Protocols, Prentice Hall, Englewood Cliffs, NJ, 1991.
URL https://spinroot.com/gerard/popd.html

[4] E. Miller, MAS: AI agent communication in Rust, source revision a61b3da98ef3ef11996dd79b4a3a048ed6c98ce1, 17 October 2026 (2026).
URL https://github.com/emmett08/mas/tree/a61b3da98ef3ef11996dd79b4a3a048ed6c98ce1

[5] T. Bray, The JavaScript Object Notation (JSON) data interchange format, RFC 8259, RFC Editor (Dec. 2017). doi:10.17487/RFC8259.
URL https://www.rfc-editor.org/rfc/rfc8259

[6] SQLite developers, Atomic commit in SQLite, accessed 17 October 2026.
URL https://www.sqlite.org/atomiccommit.html

[7] SQLite developers, Write-ahead logging, accessed 17 October 2026.
URL https://www.sqlite.org/wal.html

[8] J. H. Saltzer, D. P. Reed, D. D. Clark, End-to-end arguments in system design, ACM Transactions on Computer Systems 2 (4) (1984) 277–288.
URL https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf

[9] A. Rundgren, B. Jordan, S. Erdtman, JSON canonicalization scheme (JCS), RFC 8785, RFC Editor, informational (Jun. 2020). doi:10.17487/RFC8785.
URL https://www.rfc-editor.org/rfc/rfc8785

[10] G. J. Holzmann, The model checker SPIN, IEEE Transactions on Software Engineering 23 (5) (1997) 279–295. doi:10.1109/32.588521.
URL https://spinroot.com/gerard/pdf/inprint/ieee97.pdf